

Nav2 전체 스택 구조 & Costmap2D

Navigation2 · TurtleBot3 · ROS 2

cchyun@gmail.com

Nav2 핵심 개념

Navigation Concepts

- **Nav2**는 ROS 2 기반의 날비게이션 프레임워크
 - **Behavior Tree Navigator**: 최상위 의사결정 계층
 - **Planner Server**: 전역 경로 계산
 - **Controller Server**: 로컬 제어 신호 계산 (경로 추종)
 - **Smoother Server**: 경로의 매끄러움 개선
 - **Behavior Server**: 복구 및 기타 행동 수행
 - **AMCL / SLAM Toolbox**: 위치 추정 및 맵 생성
 - **Costmap 2D**: 환경 표현

- Action Server

- 낭비 작업(예: 날비게이션)을 제어하는 ROS 표준 인터페이스
- 요청(Request) → 피드백(Feedback) → 결과(Result)
- Nav2의 BT Navigator는 `NavigateToPose` 액션으로 통신

- Lifecycle Node & Bond

- 노드의 시작/종료를 상태 머신으로 관리
- `Unconfigured → Configuring → Inactive → Activating → Active`
- **Bond**: 서버가 살아있음을 지속적으로 알리며, 크래시 시 안전 종료

- Behavior Tree는 복잡한 로봇 작업의 트리 구조 흐름
 - 유한 상태 머신(FSM)보다 확장성이 뛰어남
 - 기본 원시 동작을 만들어 여러 행동에 **재사용**
 - Nav2는 **BehaviorTree.CPP V4** 라이브러리 사용
 - XML 파일로 트리를 정의하고, 등록된 이름으로 노드를 연결
 - `NavigateToPoseAction` 플러그인으로 외부 클라이언트 호출 가능

Behavior Tree 예시(navigate_to_pose_w_replanning_and_recovery.xml)

```
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <RecoveryNode number_of_retries="6" name="NavigateRecovery">
      <PipelineSequence name="NavigateWithReplanning">
        <RateController hz="1.0">
          <RecoveryNode number_of_retries="1" name="ComputePathToPose">
            <ComputePathToPose goal="{goal}" path="{path}" planner_id="GridBased"/>
            <ClearEntireCostmap name="ClearGlobalCostmap-Context" service_name="global_costmap/clear_entirely_global_costmap"/>
          </RecoveryNode>
        </RateController>
        <RecoveryNode number_of_retries="1" name="FollowPath">
          <FollowPath path="{path}" controller_id="FollowPath"/>
          <ClearEntireCostmap name="ClearLocalCostmap-Context" service_name="local_costmap/clear_entirely_local_costmap"/>
        </RecoveryNode>
      </PipelineSequence>
      <ReactiveFallback name="RecoveryFallback">
        <GoalUpdated/>
        <RoundRobin name="RecoveryActions">
          <Sequence name="ClearingActions">
            <ClearEntireCostmap name="ClearLocalCostmap-Subtree" service_name="local_costmap/clear_entirely_local_costmap"/>
            <ClearEntireCostmap name="ClearGlobalCostmap-Subtree" service_name="global_costmap/clear_entirely_global_costmap"/>
          </Sequence>
          <Spin spin_dist="1.57"/>
          <Wait wait_duration="5"/>
          <BackUp backup_dist="0.30" backup_speed="0.05"/>
        </RoundRobin>
      </ReactiveFallback>
    </RecoveryNode>
  </BehaviorTree>
</root>
```

Behavior Tree 상세 분석

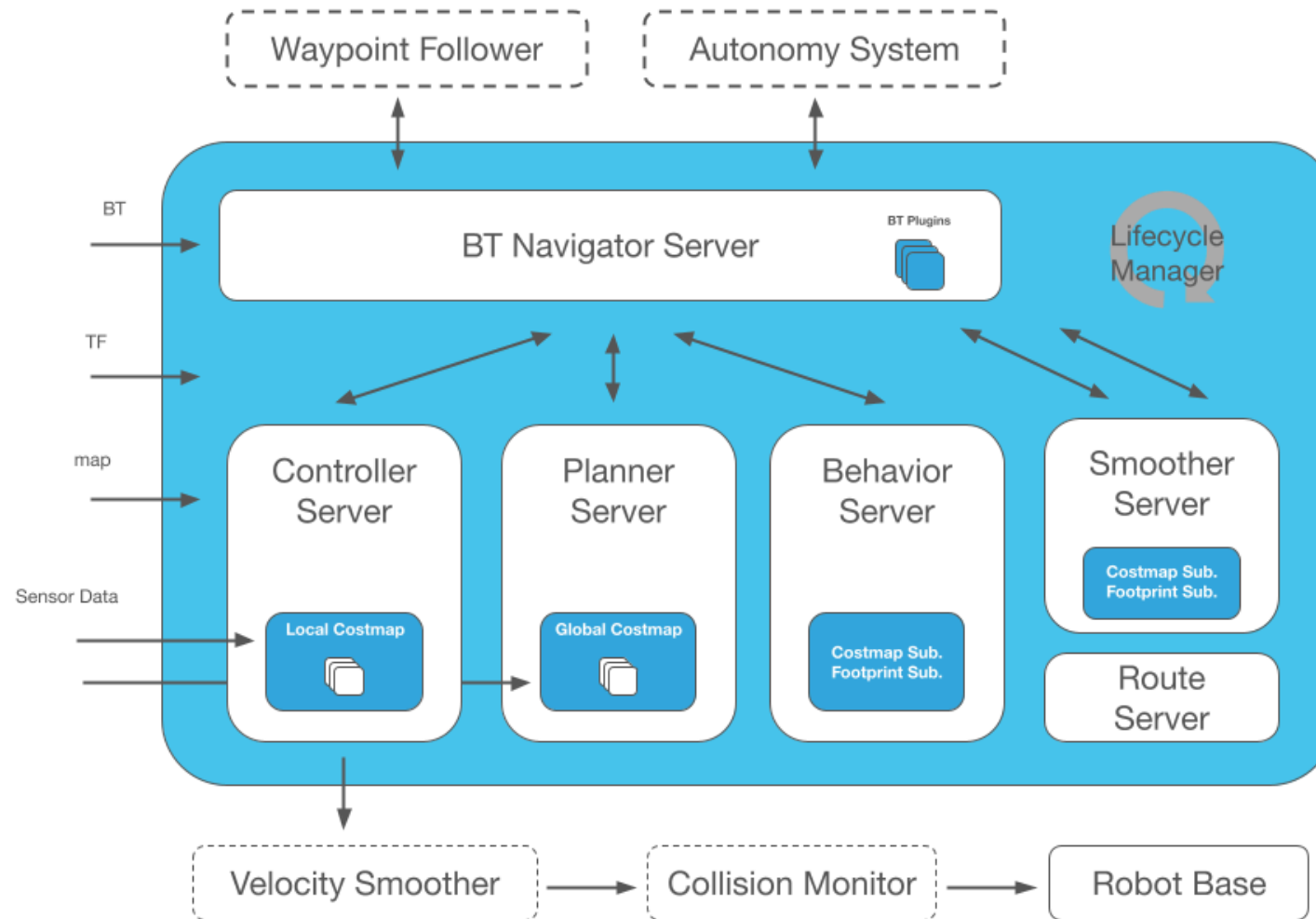
- **RecoveryNode** (최상위, 최대 6회 재시도)
 - **NavigateWithReplanning** 파이프라인이 실패하면 **RecoveryFallback** 실행
- **PipelineSequence**
 - **RateController(1Hz)**: 1초마다 경로 재계산
 - **ComputePathToPose**: **GridBased** 플래너로 전역 경로 생성 → 실패 시 **ClearGlobalCostmap**
 - **FollowPath**: **FollowPath** 컨트롤러로 경로 추적 → 실패 시 **ClearLocalCostmap**
- **ReactiveFallback** (복구 동작)
 - **GoalUpdated**: 목표가 변경되면 즉시 복구 중단
 - **RoundRobin** 순환 전략
 - 로컬/전역 코스트맵 클리어
 - 회전 (**spin_dist="1.57"**)
 - 대기 (**wait_duration="5"**)
 - 후진 (**backup_dist="0.30"**)

- 5가지 주요 액션 서버

서버	역할
Planner Server	전역 경로 계산
Controller Server	로컬 제어 신호 계산
Smoother Server	경로 매끄러움 개선
Behavior Server	복구 및 기타 행동
Route Server	내비게이션 그래프 기반 경로

- **pluginlib**을 통해 런타임에 알고리즘 플러그인 등록
- 동일 서버 내 플러그인이 **Costmap2D**를 공유

Navigation Servers



- `map → odom → base_link`
 - `map → odom`: 전역 위치 추정 (GPS, SLAM, Motion Capture)
 - `odom → base_link`: 오도메트리 (휠 엔코더, IMU 등)
 - `base_link` 이하: URDF 정적 변환
- Nav2에서 제공하는 도구
 - AMCL: 정적 맵에서 파티클 필터 기반 위치 추정
 - SLAM Toolbox: 위치 추정과 맵 생성 동시 수행
 - Robot Localization: 다중 센서 융합으로 부드러운 오도메트리 출력

- **Costmap 2D** — Nav2의 환경 표현
 - 2D 격자(Grid) 형태의 셀로 구성
 - 각 셀은 **unknown, free, occupied, inflated cost** 중 하나
 - **Costmap Layers**: LIDAR, RADAR, 소나, Depth 이미지 등의 센서 데이터를 pluginlib 플러그인으로 수집 및 병합
 - 각 레이어가 독립적으로 센서 데이터를 **구독 → 처리 → 코스트맵에 반영**
 - 여러 레이어 출력은 **Max 병합**으로 최종 맵 생성
 - **Costmap Filters**: 진입 금지 구역, 속도 제한 영역 등 필터 마스크로 행동 변경
- **기타 환경 표현**
 - Gradient Maps, 3D Costmaps, Mesh Maps, Vector Space 등

Part 1. Nav2 전체 스택 구조 개요

- 사용자가 목적지 좌표(Pose)만 입력하면 로봇이 스스로 경로를 찾아 안전하게 이동
 - Nav2 없이 자율주행을 구현할 때의 어려움
 - 오차의 누적: "앞으로 2m 가라"고 명령해도 바퀴의 미끄러짐 때문에 실제로는 1.9m만 가거나 옆으로 휠 수 있음
 - 장애물 대응: 이동 경로 중간에 갑자기 의자가 놓인다면? Nav2가 없다면 로봇은 의자를 밀고 가려다 멈추거나 넘어짐
 - 길 찾기: 방의 구조가 복잡할 때 최적 경로를 결정하는 복잡한 수학적 알고리즘을 매번 직접 계산해야 함
 - Nav2 = 로봇 자율주행 종합 패키지
 - 복잡한 기능들이 미리 구현된 강력한 프레임워크

- Nav2는 여러 독립적인 서버(Server)들이 ROS 2 Action으로 협력하여 동작 (1/2)
 - **planner_server — 글로벌 경로 계획**
 - 로봇이 현재 위치에서 목적지까지 가는 '큰 지도상의 경로'를 생성
 - 터틀봇3가 방 끝에서 반대편 끝으로 갈 때, 벽을 피해 전체적인 경로를 결정
 - `ComputePathToPose` 액션 사용
 - **controller_server — 로컬 경로 추종 및 장애물 회피**
 - 플래너가 만든 큰 경로를 따라가면서, 바로 앞에 있는 장애물을 실시간으로 회피
 - 경로 상에 갑자기 나타난 가방을 피하기 위해 바퀴 속도를 조절
 - `FollowPath` 액션 사용

- Nav2는 여러 독립적인 서버(Server)들이 ROS 2 Action으로 협력하여 동작 (2/2)
 - **recoveries_server — 회복 행동**
 - 로봇이 장애물에 갇혀 오도 가도 못할 때 해결책을 제시
 - **Spin**: 제자리에서 돌아 공간을 확보
 - **BackUp**: 뒤로 살짝 물러나 장애물에서 벗어남
 - **Wait**: 사람이 지나갈 때까지 잠시 대기
 - **map_server**
 - 미리 만들어진 **사전 지도(Static Map)** 를 읽어와 다른 노드들에게 제공
 - **amcl — 위치 추정**
 - 로봇의 LiDAR 데이터와 지도를 비교하여, 현재 로봇이 지도상 어디에 있는지 확률적으로 계산
 - 눈을 감고 방 안을 더듬으며 자신의 위치를 맞추는 것과 유사

- 로봇 자율주행에는 여러 좌표계(Frame) 간의 관계를 정의하는 TF(Transform) 체인이 필수
 - `map → odom` (AMCL 제공)
 - 지도의 원점과 주행 기록(Odometry) 원점 사이의 관계
 - AMCL은 바퀴 오차로 인한 위치의 미세한 틀어짐을 지도를 보고 교정
 - `odom → base_link` (Odometry 시스템 제공)
 - 로봇이 출발한 지점(odom)으로부터 바퀴가 얼마나 굴러갔는지 계산
 - 모터 드라이버나 센서 퓨전 노드가 담당
 - `base_link → 센서 프레임` (robot_state_publisher 제공)
 - 로봇 중심에서 LiDAR나 카메라가 어디에 달려 있는지 고정된 정보
 - 로봇 설계도(URDF 파일) 기반으로 제공

- 터틀봇3가 목적지까지 이동하는 전체 프로세스

1. 목표 지점 입력: 사용자가 목적지 좌표를 주면 `NavigateToPose` 액션이 시작
2. 글로벌 경로 계획: `planner_server`가 전체 지도를 보고 목적지까지의 경로를 생성
3. 로컬 경로 추종: `controller_server`가 생성된 경로를 보고 바퀴에 속도 명령(`cmd_vel`)을 전달
4. 장애물 감지: 이동 중 LiDAR가 장애물을 발견하면 `controller_server`가 즉시 경로를 수정
5. 목적지 도달: 로봇이 목표 지점의 허용 범위(Tolerance) 안에 들어오면 주행 완료

- Nav2를 실행하기 위해 최소한 다음 노드들이 필요

노드	역할
map_server	사전 지도 로드
amcl	현재 위치 추정
planner_server	글로벌 경로 계획
controller_server	바퀴 제어
nav2_lifecycle_manager	위 모든 노드의 상태 관리

- lifecycle_manager가 없으면 각 노드가 실행되더라도 제대로 동작하지 않음

- Nav2의 모든 노드는 Lifecycle Node(관리형 노드)로 구현되며, lifecycle_manager가 이를 관리
 - 순서 관리
 - 지도가 먼저 로드 → AMCL이 위치 파악 → 플래너가 길 탐색
 - 올바른 실행 순서를 보장하여 안정적인 시작 확보
 - 상태 전이
 - 노드들을 `Unconfigured` → `Inactive` → `Active` 상태로 단계별로 전환
 - 모든 설정이 완료된 후에만 로봇이 움직이도록 보장
 - 안전성(Bond)
 - 주행 중 핵심 노드 중 하나가 비정상 종료되면, 관리자가 이를 감지
 - 전체 시스템을 안전하게 정지시키거나 재시작

- "왜 플래너와 코스트맵이 두 개씩 있나요?"
 - 글로벌: 전체 지도 기반 '길 찾기용'
 - 로컬: 바로 앞의 '장애물 회피용'
- "노드가 실행 중인데 왜 아무 반응이 없나요?"
 - `lifecycle_manager`가 노드를 `Active` 상태로 전환해 주지 않으면
 - 노드가 실행 중이어도 아무런 기능을 수행하지 않음
 - Nav2 입문 시 가장 생소하고 자주 빠지는 함정

Part 2. Costmap2D 구조 & 파라미터

Costmap이란 무엇인가?

- Costmap = 로봇 주변 환경을 2D 격자(Grid)로 표현한 지도
 - 각 셀에는 해당 지점을 통과할 때 발생하는 "비용(Cost)" 값이 저장
 - 사전 지도(Static Map)의 벽 위치 + LiDAR 실시간 장애물 정보를 합산
 - 비용 값의 의미

값	의미
0	자유 공간(Free) — 안전하게 통과 가능
1 ~ 253	인플레이션 영역 — 장애물 근처 위험 구간
254	치명적 장애물(Lethal) — 절대 통과 불가
255	알 수 없는 영역(Unknown)

- 로봇은 "비용이 높은 곳은 위험하니 가지 말아야 한다"고 판단

- 로봇은 두 가지 목적으로 서로 다른 Costmap을 별도로 관리

구분	글로벌 Costmap	로컬 Costmap
주요 용도	전체 지도 기반 장기 경로 계획	로봇 주변 장애물 회피 및 단기 주행
기반 데이터	사전 정적 지도 + 광범위 센서	로봇 주변 실시간 센서 데이터
크기 기준	전체 주행 영역을 포함할 만큼 크게 설정	Rolling Window로 좁게 설정 (예: 5m × 5m)
해상도	계산 효율을 위해 다소 낮게 가능 (예: 0.1m)	정밀한 회피를 위해 높게 권장 (예: 0.05m)

Costmap 레이어(Layer) 구조

- Nav2는 여러 층의 레이어를 겹쳐서 최종 Costmap을 생성
 - StaticLayer
 - SLAM 등으로 미리 만든 **사전 지도의 장애물** 정보를 반영
 - 정적인 벽이나 기둥을 표현하는 기초 레이어
 - ObstacleLayer
 - LiDAR나 카메라 센서를 통해 실시간으로 들어오는 **동적 장애물**을 반영
 - 장애물이 나타나면 지도에 **표시(Marking)**, 사라지면 **제거(Clearing)**
 - InflationLayer
 - 장애물 주변에 **비용을 퍼뜨려(Inflate)** 로봇이 충분한 거리를 유지하며 주행하도록 유도
 - 로봇의 기하학적 크기를 고려한 안전 거리를 확보하는 역할

- **inflation_radius** — 장애물로부터 비용을 부풀릴 반경
 - 로봇의 반지름보다 크게 설정해야 하는 이유
 - 로봇 중심이 장애물에 닿지 않더라도 로봇의 몸체가 벽에 부딪히는 것을 방지
 - 경로 계획 알고리즘이 장애물을 여유 있게 우회하도록 유도
- **cost_scaling_factor** — 장애물에서 멀어질수록 비용이 감소하는 기울기

값 설정	동작 결과
값이 작을수록	비용이 천천히 감소 → 로봇이 장애물에서 멀리 떨어져 주행
값이 클수록	비용이 급격히 감소 → 로봇이 장애물에 더 가깝게 붙어 주행, 너무 크면 좁은 길 통과 불가

- **update_frequency** — 지도를 센서 데이터에 맞춰 다시 계산하는 빈도(Hz)
 - 로봇의 이동 속도가 빠를수록 높게 설정 필요
 - CPU 부하를 고려하여 적절히 조절 (일반적으로 5Hz 권장)
- **publish_frequency** — 계산된 지도를 RViz2로 시각화하기 위해 발행하는 빈도(Hz)
 - 시각적 확인이 중요하지 않다면 update_frequency보다 낮게 설정하여 대역폭 절약 가능
- **글로벌 vs 로컬 권장 설정**

	update_frequency	이유
Global Costmap	1.0 Hz	전체 경로는 빈번한 갱신 불필요
Local Costmap	5.0 Hz	실시간 장애물 회피를 위해 빠른 갱신 필요

- Costmap을 RViz2에서 직관적으로 확인하는 절차

1. RViz2 실행 후 왼쪽 아래 'Add' 버튼 클릭

2. 'By topic' 탭에서 토픽 선택

종류	토픽 경로
글로벌 Costmap	<code>/global_costmap/costmap</code>
로컬 Costmap	<code>/local_costmap/costmap</code>

3. 추가된 레이어의 'Color Scheme' 속성을 **costmap** 으로 설정

- 비용에 따른 색상 변화를 직관적으로 확인 가능

- 장애물이 반영 안 됨

- 센서 토픽 이름이 올바른지 확인
- `max_obstacle_height` 가 센서의 높이보다 낮게 설정되지 않았는지 점검 (3D 전용)

- 벽에 너무 가깝게 주행

- `inflation_radius` 를 키워 장애물 주변의 안전 거리를 더 넓게 확보
- `cost_scaling_factor` 값을 낮게 조절하여 비용 감소 기울기를 완만하게 설정

- 좁은 공간 통과 불가

- 로봇의 실제 크기(`robot_radius`) 설정이 너무 큰지 확인
- 인플레이션 반경이 과도하게 설정되어 통로 전체가 높은 비용으로 채워졌는지 점검

기본 Costmap YAML 예시 (글로벌)

```
global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0          # Costmap 갱신 주기 (Hz). 1초에 1번 장애물 정보를 업데이트
      publish_frequency: 1.0         # Costmap 발행 주기 (Hz). 1초에 1번 /global_costmap 토픽 발행
      global_frame: map              # Costmap의 기준 좌표계. SLAM이나 AMCL의 map 프레임 사용
      robot_base_frame: base_link    # 로봇의 기준 프레임. 로봇 본체 중심 좌표계
      use_sim_time: True              # Gazebo 시뮬레이션 시간 사용 여부
      robot_radius: 0.22             # 로봇의 반지름 (미터). 원형 로봇 가정 시 장애물과의 최소 간격 계산에 사용
      resolution: 0.05               # Costmap 격자 해상도 (m/cell). 5cm당 1픽셀. 값이 작을수록 정밀하지만 연산량 증가
      track_unknown_space: true       # 미탐색 영역(unknown)을 별도로 추적. 경로 계획 시 탐색 유도에 활용
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"] # 적용할 레이어 목록 (순서대로 처리)
```

기본 Costmap YAML 예시 (글로벌-계속)

```
obstacle_layer:
  plugin: "nav2_costmap_2d::ObstacleLayer" # 실시간 센서 데이터로 동적 장애물을 감지하는 레이어
  enabled: True
  observation_sources: scan # 사용할 센서 소스 목록. 여기서는 LiDAR 스캔만 사용
  scan:
    topic: /scan # 구독할 스캔 토픽 이름
    max_obstacle_height: 2.0 # 장애물로 인식할 최대 높이 (m). 2m 이상은 무시 (예: 천장)
    clearing: True # 레이 트레이싱으로 장애물이 사라진 영역을 지우는 기능 활성화
    marking: True # 스캔으로 감지된 점을 장애물로 표시하는 기능 활성화
    data_type: "LaserScan" # 센서 데이터 타입. LaserScan 메시지 사용
    raytrace_max_range: 3.0 # 레이 트레이싱 최대 거리 (m). 3m 넘어가면 장애물 제거 판단 안 함
    raytrace_min_range: 0.0 # 레이 트레이싱 최소 거리 (m). 너무 가까운 데이터 무시
    obstacle_max_range: 2.5 # 장애물 표시 최대 거리 (m). 2.5m까지만 장애물로 기록
    obstacle_min_range: 0.0 # 장애물 표시 최소 거리 (m)
  static_layer:
    plugin: "nav2_costmap_2d::StaticLayer" # SLAM/AMCL에서 제공하는 정적 지도를 불러오는 레이어
    map_subscribe_transient_local: True # 지도를 한 번만 수신하고 지속 보관 ( latch 모드 )
  inflation_layer:
    plugin: "nav2_costmap_2d::InflationLayer" # 장애물 주변에 안전 버퍼(비용)를 추가하는 레이어
    cost_scaling_factor: 3.0 # 비용 감소율. 값이 클수록 장애물에서 멀리 떨어질수록 비용이 빠르게 줄어듦
    inflation_radius: 0.55 # 팽창 반경 (m). 장애물에서 55cm 이내 영역에 비용(충돌 위험) 부여
  always_send_full_costmap: True # 매번 전체 Costmap 메시지를 발행. False면 변경된 부분만 발행
```

기본 Costmap YAML 예시 (로컬)

```
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 5.0      # Costmap 갱신 주기 (Hz). 로컬은 글로벌보다 빠르게(5Hz) 갱신하여 즉각 반응
      publish_frequency: 2.0     # Costmap 발행 주기 (Hz). 0.5초마다 /local_costmap 토픽 발행
      global_frame: odom         # 로컬 Costmap 기준 좌표계. odom 사용 (짧은 구간에서는 drift가 적음)
      robot_base_frame: base_link # 로봇의 기준 프레임
      use_sim_time: True         # Gazebo 시뮬레이션 시간 사용 여부
      rolling_window: true       # 로봇을 중심으로 창이 이동하는 롤링 윈도우 모드. 무한한 지도 대신 주변만 관리
      width: 3                   # Costmap 가로 크기 (미터). 로봇 중심 좌우 1.5m씩, 총 3m 폭의 지역만 관리
      height: 3                  # Costmap 세로 크기 (미터). 로봇 중심 앞뒤 1.5m씩, 총 3m 높이의 지역만 관리
      resolution: 0.05           # Costmap 격자 해상도 (m/cell). 글로벌과 동일하게 5cm
      robot_radius: 0.22         # 로봇의 반지름 (미터)
      plugins: ["voxel_layer", "inflation_layer"] # 적용할 레이어 목록. 3D 볼륨 감지용 VoxelLayer 사용
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer" # 장애물 주변 안전 버퍼(비용) 레이어
        cost_scaling_factor: 3.0 # 비용 감소율. 값이 클수록 장애물에서 멀어질수록 비용이 빠르게 줄어듦
        inflation_radius: 0.55 # 팽창 반경 (m). 장애물에서 55cm 이내 영역에 비용 부여
```



```
voxel_layer:
  plugin: "nav2_costmap_2d::VoxelLayer" # 3D 공간을 복셀(격자)로 나눠 장애물을 입체적으로 감지하는 레이어
  enabled: True
  publish_voxel_map: True # 3D 복셀 맵을 토픽으로 발행 (디버깅/시각화용)
  origin_z: 0.0 # 복셀 그리드의 Z축 기준 높이 (m). 지면(0m)부터 시작
  z_resolution: 0.05 # Z축(높이) 방향 복셀 크기 (m). 5cm 단위로 수직 분할
  z_voxels: 16 # Z축 방향 복셀 개수. 총 16개 × 5cm = 80cm 높이까지 3D 공간 관리
  max_obstacle_height: 2.0 # 장애물로 인식할 최대 높이 (m). 2m 이상은 무시
  mark_threshold: 0 # 복셀이 장애물로 표시되기 위한 최소 점유 횟수 임계값. 0이면 한 번 감지 시 즉시 표시
  observation_sources: scan # 사용할 센서 소스 목록
  scan:
    topic: /scan # 구독할 스캔 토픽 이름
    max_obstacle_height: 2.0 # 해당 소스에서 인식할 최대 장애물 높이 (m)
    clearing: True # 레이 트레이싱으로 장애물이 사라진 영역을 지우는 기능 활성화
    marking: True # 스캔으로 감지된 점을 장애물로 표시하는 기능 활성화
    data_type: "LaserScan" # 센서 데이터 타입
    raytrace_max_range: 3.0 # 레이 트레이싱 최대 거리 (m)
    raytrace_min_range: 0.0 # 레이 트레이싱 최소 거리 (m)
    obstacle_max_range: 2.5 # 장애물 표시 최대 거리 (m)
    obstacle_min_range: 0.0 # 장애물 표시 최소 거리 (m)
  always_send_full_costmap: True # 매번 전체 Costmap 메시지를 발행
```